

Zcash block-download scheduler specification.

Models the inventory-routing registry and download pipeline that sit one layer above the per-connection protocol in *protocol.tla*. Where *protocol.tla* models a single connection pulling blocks from one partner, this module models the node choosing which of many peers to ask for each block — the layer where Zebra’s genesis-to-tip sync stall lived (*ZcashFoundation/zebra* $\neq$ 10679, symptom $\neq$ 5709).

A local syncer downloads blocks $1 \dots \text{MaxBlock}$ from a pool of peers. Each peer genuinely holds some blocks (Holders); requesting a block a peer does not have yields an explicit notfound, while any request may also fail transiently (timeout or dropped connection). The scheduler keeps an availability registry and only routes a getdata to a peer not already marked missing for that block.

Two booleans select Zebra’s pre- and post-fix behaviour:

BuggyRouting = TRUE $\rightarrow$ a transient error marks the peer missing for the block, poisoning the routing registry.
 = FALSE $\rightarrow$ a transient error leaves the entry unknown.

BuggyDrop = TRUE $\rightarrow$ a notfound silently drops the block from the work set, wedging the frontier.
 = FALSE $\rightarrow$ a notfound re-queues the block (bounded retries), routed away from the notfound peer.

The safety invariant *RegistryHonest* and the liveness property *EventuallyAllVerified* both hold under the fixed behaviour and are violated under the buggy behaviour, reproducing the stall as a *TLC* counterexample.

See documents/sync-stall-modeling.md for the full write-up.

EXTENDS *Naturals*, *FiniteSets*

CONSTANT <i>Peers</i>	set of peer ids
CONSTANT <i>MaxBlock</i>	blocks to download are $1 \dots \text{MaxBlock}$
CONSTANT <i>LaggingPeers</i>	peers that are behind: they hold only blocks $1 \dots \text{LagTip}$
CONSTANT <i>LagTip</i>	chain height of a lagging peer ($< \text{MaxBlock}$)
CONSTANT <i>MaxRetries</i>	bound on re-queue attempts per block after a notfound
CONSTANT <i>BuggyRouting</i>	TRUE reproduces the registry-poisoning bug
CONSTANT <i>BuggyDrop</i>	TRUE reproduces the silent-drop bug

Blocks $\triangleq 1 \dots \text{MaxBlock}$

NONE $\triangleq$ “none” sentinel for “no peer”; must not be a member of *Peers*

Ground truth: a lagging peer holds blocks up to *LagTip*; every other peer is at tip and holds them all. The scheduler never reads this directly — it learns a peer’s contents only through deliver/notfound. A degraded peer set (most peers lagging) is what made the stall visible in practice.

PeerTip(*p*) $\triangleq$ IF *p* $\in$ *LaggingPeers* THEN *LagTip* ELSE *MaxBlock*

Holds(*p*, *b*) $\triangleq b \leq \text{PeerTip}(p)$

VARIABLES

<i>verified</i> ,	subset of <i>Blocks</i> downloaded and verified (the frontier)
<i>pending</i> ,	subset of <i>Blocks</i> the scheduler still wants and may request

<i>inflight</i> ,	[<i>Blocks</i> $\rightarrow$ <i>Peers</i> $\cup$ { <i>NONE</i> }] peer a block is requested from
<i>avail</i> ,	[<i>Peers</i> $\rightarrow$ [<i>Blocks</i> $\rightarrow$ { "has", "missing", "unknown" }]] registry
<i>retries</i>	[<i>Blocks</i> $\rightarrow$ 0 .. <i>MaxRetries</i>] re-queue attempts used

vars $\triangleq$ $\langle \textit{verified}, \textit{pending}, \textit{inflight}, \textit{avail}, \textit{retries} \rangle$

Init $\triangleq$

$\wedge \textit{verified} = \{\}$
 $\wedge \textit{pending} = \textit{Blocks}$
 $\wedge \textit{inflight} = [b \in \textit{Blocks} \mapsto \textit{NONE}]$
 $\wedge \textit{avail} = [p \in \textit{Peers} \mapsto [b \in \textit{Blocks} \mapsto \text{"unknown"}]]$
 $\wedge \textit{retries} = [b \in \textit{Blocks} \mapsto 0]$

The scheduler routes a getdata for a pending block to a peer not already marked missing for it (peer-to-candidate choice over the un-poisoned peers).

Request $\triangleq$

$\exists b \in \textit{pending} :$
 $\exists p \in \textit{Peers} :$
 $\wedge \textit{inflight}[b] = \textit{NONE}$
 $\wedge \textit{avail}[p][b] \neq \text{"missing"}$
 $\wedge \textit{inflight}' = [\textit{inflight} \text{ EXCEPT } ![b] = p]$
 $\wedge \textit{pending}' = \textit{pending} \setminus \{b\}$
 $\wedge \text{UNCHANGED } \langle \textit{verified}, \textit{avail}, \textit{retries} \rangle$

The in-flight peer genuinely has the block: it is delivered and verified, and the registry correctly records that the peer has it.

DeliverBlock $\triangleq$

$\exists b \in \textit{Blocks} :$
 $\wedge \textit{inflight}[b] \neq \textit{NONE}$
 $\wedge \textit{Holds}(\textit{inflight}[b], b)$
 $\wedge \textit{verified}' = \textit{verified} \cup \{b\}$
 $\wedge \textit{avail}' = [\textit{avail} \text{ EXCEPT } ![\textit{inflight}[b]][b] = \text{"has"}]$
 $\wedge \textit{inflight}' = [\textit{inflight} \text{ EXCEPT } ![b] = \textit{NONE}]$
 $\wedge \text{UNCHANGED } \langle \textit{pending}, \textit{retries} \rangle$

The in-flight peer genuinely lacks the block and says so. Marking the peer missing for this block is correct — an explicit notfound. What the scheduler does with the block then is the *BuggyDrop* switch: drop it, or re-queue it (under the retry bound) so it is routed to a different peer next time.

NotFound $\triangleq$

$\exists b \in \textit{Blocks} :$
 $\text{LET } p \triangleq \textit{inflight}[b] \text{ IN}$
 $\wedge p \neq \textit{NONE}$
 $\wedge \neg \textit{Holds}(p, b)$
 $\wedge \textit{inflight}' = [\textit{inflight} \text{ EXCEPT } ![b] = \textit{NONE}]$

$$\begin{aligned}
& \wedge \textit{avail}' = [\textit{avail} \text{ EXCEPT } ![p][b] = \text{"missing"}] \\
& \wedge \text{IF } \neg \textit{BuggyDrop} \wedge \textit{retries}[b] < \textit{MaxRetries} \\
& \quad \text{THEN } \wedge \textit{pending}' = \textit{pending} \cup \{b\} \\
& \quad \quad \wedge \textit{retries}' = [\textit{retries} \text{ EXCEPT } ![b] = @ + 1] \\
& \quad \text{ELSE } \wedge \text{UNCHANGED } \textit{pending} \quad \text{silent drop (buggy), or retries exhausted} \\
& \quad \quad \wedge \text{UNCHANGED } \textit{retries} \\
& \wedge \text{UNCHANGED } \textit{verified}
\end{aligned}$$

Any in-flight request may fail transiently (timeout or dropped connection), independent of whether the peer has the block. The request is abandoned and the block re-queued; the *BuggyRouting* switch decides the registry entry — poisoned (missing) or left unknown.

$$\begin{aligned}
\textit{TransientError} & \triangleq \\
& \exists b \in \textit{Blocks} : \\
& \quad \text{LET } p \triangleq \textit{inflight}[b] \text{ IN} \\
& \quad \wedge p \neq \textit{NONE} \\
& \quad \wedge \textit{inflight}' = [\textit{inflight} \text{ EXCEPT } ![b] = \textit{NONE}] \\
& \quad \wedge \textit{pending}' = \textit{pending} \cup \{b\} \\
& \quad \wedge \textit{avail}' = [\textit{avail} \text{ EXCEPT } ![p][b] = \text{IF } \textit{BuggyRouting} \text{ THEN "missing" ELSE "unknown"}] \\
& \quad \wedge \text{UNCHANGED } \langle \textit{verified}, \textit{retries} \rangle
\end{aligned}$$

$$\begin{aligned}
\textit{Next} & \triangleq \\
& \vee \textit{Request} \\
& \vee \textit{DeliverBlock} \\
& \vee \textit{NotFound} \\
& \vee \textit{TransientError}
\end{aligned}$$

Strong fairness on the progress actions encodes the real-world assumption that the node keeps trying and a willing peer eventually responds. The transient error gets no fairness — it is adversarial but never forced.

$$\begin{aligned}
\textit{Spec} & \triangleq \\
& \textit{Init} \\
& \wedge \Box [\textit{Next}]_{\textit{vars}} \\
& \wedge \text{SF}_{\textit{vars}}(\textit{Request}) \\
& \wedge \text{SF}_{\textit{vars}}(\textit{DeliverBlock}) \\
& \wedge \text{SF}_{\textit{vars}}(\textit{NotFound})
\end{aligned}$$

Liveness: every block is eventually downloaded and verified. The genesis-to-tip stall is a violation of this property.

$$\textit{EventuallyAllVerified} \triangleq \Diamond(\textit{verified} = \textit{Blocks})$$

Safety invariants.

$$\begin{aligned}
TypeOK &\triangleq \\
&\wedge \text{ verified } \subseteq Blocks \\
&\wedge \text{ pending } \subseteq Blocks \\
&\wedge \text{ inflight } \in [Blocks \rightarrow Peers \cup \{NONE\}] \\
&\wedge \text{ avail } \in [Peers \rightarrow [Blocks \rightarrow \{\text{"has"}, \text{"missing"}, \text{"unknown"}\}]] \\
&\wedge \text{ retries } \in [Blocks \rightarrow 0 \dots MaxRetries]
\end{aligned}$$

The registry never marks a peer missing for a block it actually holds.
This is the formal statement of “a timeout is not a notfound”: under *BuggyRouting* a transient error on a holder violates it immediately.

$$\begin{aligned}
RegistryHonest &\triangleq \\
&\forall p \in Peers : \\
&\quad \forall b \in Blocks : \\
&\quad \quad \text{avail}[p][b] = \text{"missing"} \Rightarrow \neg Holds(p, b)
\end{aligned}$$

A peer is only ever recorded as having a block it genuinely holds.

$$\begin{aligned}
RegistryHasIsSound &\triangleq \\
&\forall p \in Peers : \\
&\quad \forall b \in Blocks : \\
&\quad \quad \text{avail}[p][b] = \text{"has"} \Rightarrow Holds(p, b)
\end{aligned}$$

A verified block was genuinely available from some peer.

$$\begin{aligned}
VerifiedAreReal &\triangleq \\
&\forall b \in verified : \\
&\quad \exists p \in Peers : Holds(p, b)
\end{aligned}$$